



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

The four PEAS components are Performance measure, Environment, Actuators, and Sensors. Performance measure judges how well the agent is doing, Environment is the world it operates in, Actuators are the actions it can take, and Sensors are how it

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

The variables representing the Environment state are `self.width` and `self.height` (grid size), `self.walls` (fixed obstacles), `self.food_positions` (food locations), and `self.opponents` (their positions). We also added `self.toxic_traps` for hidden hazards. These together describe the physical world independent of the agent's own position.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

This is a Multi-Agent environment. The `self.opponents` list moves on its own using `random.choice(['Up', 'Down', 'Left', 'Right', 'Stay'])` inside `execute_action()`, completely independent of the player. Since multiple entities are acting and affecting each other (e.g. opponent colliding with the agent causes a -50 penalty), this counts as multi-

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete, ordered history of everything the agent has perceived from when it started until now, not just its current observation. It matters because an agent's decisions can depend on this full history rather than just the

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

`get_percept()` represents the Sensors component. It gives the agent information about the environment, like its position, opponent positions, and whether it hit a wall, found food, or is on a trap. This is the only channel through which the agent

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

This environment is Partially Observable. The percept dictionary only reveals the agent's own position, the opponents' current positions, and the status of the single cell the agent is standing on. It never reveals the full list of food, wall, or trap locations elsewhere on the grid, so the agent never sees the complete world state at

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance measure is being updated. Hitting a wall triggers `self.score -= 5`, and since `self.score` is what judges the agent's performance, any change to it is a direct performance measure update.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Rationality is judged by actual outcomes in the environment, not by how much internal effort or computation the agent used. If we measured internal processing instead, an agent could get a good score by doing minimal thinking while still behaving badly (like repeatedly hitting walls). Measuring external results keeps the

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

- Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Hiding `self.toxic_traps` from the sensors reinforces that the environment is Partially Observable. Even with the `smells_toxin` sensor, the agent only detects a trap once it's already standing on that cell, with no way to see other trap locations in advance. This

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

The smells_toxin sensor tells the agent whether its current cell has a trap, giving it more information to base decisions on. This lets a smarter agent avoid returning to that cell and reduce unnecessary score loss from the -15 penalty. That directly supports rationality, since a rational agent picks actions that maximize expected

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic example is a vacuum agent whose performance measure is just "total dirt cleaned." The agent can exploit this by sucking up dirt, dumping it back in the same spot, and cleaning it again repeatedly, racking up infinite score without doing real

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!



FACULTY OF COMPUTING

